DOCUMENTACIÓN DE LA APLICACIÓN

AutoParts

MANUAL TÉCNICO

Arquitectura, instalación, funcionamiento, datos y mantenimiento

Versión de referencia: octubre de 2026

Proyecto académico: tienda de repuestos automotrices

1. Introducción y alcance

AutoParts es una aplicación móvil Android para una tienda de repuestos automotrices. Permite que clientes consulten el catálogo, busquen artículos, gestionen un carrito y registren pedidos; también dispone de un módulo de administración para gestionar productos, inventario y estados de pedidos.

Este documento describe la implementación académica local comprobada. No debe interpretarse como una guía de despliegue público o de seguridad de producción.

2. Objetivos y módulos

- Cliente: registro, inicio/cierre de sesión, consulta y búsqueda de productos, filtrado por categoría, detalles, carrito, confirmación e historial de pedidos.
- Administrador: acceso por rol, panel de inventario, catálogo, alta/edición de productos, activación/desactivación y gestión de pedidos.
- Persistencia: mantener usuarios de aplicación, catálogo, existencias y pedidos en MySQL mediante una API REST.

3. Herramientas y tecnologías

Tecnología	Uso en el proyecto
Android Studio	Entorno de desarrollo, compilación y pruebas Android
Kotlin	Lógica de Activities, validaciones y navegación
XML / Android Views	Diseño de pantallas y componentes de interfaz
Firebase Authentication	Creación de cuentas e inicio de sesión con correo y contraseña
Node.js + Express	Servidor backend y endpoints REST
MySQL Server	Almacenamiento de datos comerciales
MySQL Workbench	Administración de esquema y consultas SQL
Retrofit + Gson	Comunicación HTTP y transformación de JSON
Kotlin Coroutines	Ejecución asíncrona de solicitudes
RecyclerView	Listas y cuadrículas de productos y pedidos
Gradle	Construcción y dependencias del proyecto

4. Arquitectura y flujo de información

Arquitectura cliente-servidor: Android (Kotlin/XML) → Retrofit/JSON → API REST (Node.js/Express) → MySQL. Firebase Authentication es un servicio separado que administra credenciales y sesiones de usuario.

1. El usuario se registra o inicia sesión mediante Firebase Authentication.
2. Android obtiene el UID de Firebase y consulta o sincroniza el registro de usuario con la API.
3. La API consulta MySQL para obtener el rol cliente/administrador y datos de la aplicación.
4. Según el rol, Android abre MainActivity (cliente) o AdminActivity (administrador).

5. Las consultas de catálogo, inventario y pedidos se envían por Retrofit a la API REST.
6. El servidor lee o modifica las tablas MySQL y devuelve respuestas JSON que Android muestra en pantalla.

Diferencia importante: Firebase Authentication conserva las credenciales y gestiona el inicio de sesión. MySQL conserva los datos comerciales y la referencia firebase_uid del usuario. Las contraseñas no se almacenan en la tabla usuarios.

5. Estructura del proyecto Android

Componente	Responsabilidad
LoginActivity / RegisterActivity	Acceso y alta de cuentas Firebase; sincronización inicial
MainActivity / ProductoAdapter	Catálogo, búsqueda y filtros de cliente
DetalleProductoActivity	Información de un repuesto y agregado al carrito
CarritoManager / CarritoActivity	Carrito en memoria y confirmación de pedido
HistorialPedidosActivity	Consulta del historial y estados
AdminActivity	Panel principal e inventario
AdminCatalogoActivity	Listado y activación/desactivación de productos
AdminProductoFormActivity	Creación y modificación de productos
AdminPedidosActivity	Consulta y cambio de estado de pedidos
ApiService / RetrofitClient	Declaración de endpoints y cliente HTTP

6. Base de datos MySQL

Esquema utilizado: autorepuestos_db. La implementación contiene seis tablas principales.

Tabla	Información principal
Usuarios	id_usuario, firebase_uid, nombre, correo, rol
Categorias	id_categoria, nombre, descripción
Productos	id_producto, id_categoria, nombre, marca, descripción, precio, imagen_url, activo
Inventario	id_inventario, id_producto, stock, stock_minimo
Pedidos	id_pedido, id_usuario, fecha_pedido, total, estado
detalle_pedidos	id_detalle, id_pedido, id_producto, cantidad, precio_unitario, subtotal

Relaciones principales: productos.id_categoria → categorias.id_categoria; inventario.id_producto → productos.id_producto; pedidos.id_usuario → usuarios.id_usuario; detalle_pedidos.id_pedido → pedidos.id_pedido y detalle_pedidos.id_producto → productos.id_producto.

El estado de pedido puede ser pendiente, completado o cancelado. El campo activo permite desactivar un producto sin borrar su historial. El inventario conserva stock y stock mínimo.

7. API REST y comunicación

URL de desarrollo utilizada en el emulador Android: http://10.0.2.2:3000/. La dirección 10.0.2.2 apunta al equipo anfitrión desde el emulador; no debe utilizarse sin ajustes en un teléfono físico.

Método / ruta	Función
POST /api/usuarios	Registrar/sincronizar datos de usuario
GET /api/usuarios/rol/:firebase_uid	Consultar rol del usuario
GET /api/productos	Consultar catálogo activo
POST /api/pedidos	Crear pedido con productos y cantidades
GET /api/pedidos/usuario/:firebase_uid	Obtener historial del cliente
GET /api/admin/inventario	Consultar inventario administrativo
GET /api/admin/categorias	Consultar categorías
POST /api/admin/productos	Crear producto
PUT /api/admin/productos/:id	Editar producto
PATCH /api/admin/productos/:id/activo	Activar/desactivar producto
GET /api/admin/pedidos	Consultar pedidos de clientes
PATCH /api/admin/pedidos/:id/estado	Completar o cancelar pedido

Los endpoints administrativos de la versión local usan el encabezado x-firebase-uid para identificar el usuario y verificar su rol en MySQL. Esta identificación no valida criptográficamente un token Firebase; para publicar el servicio en Internet sería necesario verificar tokens ID en el servidor, autorizar por rol y reforzar la seguridad.

8. Procesos funcionales

8.1 Registro e inicio de sesión

RegisterActivity valida nombre, correo, longitud de contraseña y confirmación. Crea la cuenta en Firebase, configura displayName y solicita el registro del cliente en MySQL con rol cliente. LoginActivity autentica con Firebase y consulta el rol almacenado en MySQL para redirigir al módulo correspondiente.

8.2 Productos e imágenes

El catálogo se obtiene de MySQL por la API. El cliente filtra por nombre, marca y categoría. La demostración actual muestra fotografías empaquetadas en res/drawable y asociadas a cinco IDs de producto (1 a 5). Por ello, la columna imagen_url puede estar vacía sin impedir que esas imágenes se muestren. Para nuevos productos sin imagen asociada se utiliza una imagen de reserva.

8.3 Carrito y pedidos

CarritoManager conserva temporalmente los artículos seleccionados, cantidades y total en memoria. Al confirmar, Android envía IDs y cantidades a POST /api/pedidos; el backend verifica existencias, calcula precios con los valores de MySQL, registra el pedido y sus detalles y descuenta stock mediante una transacción. Un carrito vacío no se confirma y muestra una advertencia.

El administrador puede marcar pedidos como completados o cancelados. En la implementación comprobada, cancelar un pedido restaura existencias mediante el servidor. El cliente visualiza el estado actualizado en su historial.

8.4 Inventario y baja lógica

Al editar un producto, el administrador puede modificar sus datos y existencias. Desactivar equivale a establecer activo=0: el artículo deja de ofrecerse en el catálogo del cliente, pero permanece en MySQL y en la administración. Reactivar vuelve a habilitar su consulta comercial.

9. Instalación y ejecución local

1. Instalar Android Studio y los componentes del SDK requeridos por el proyecto.
2. Instalar Node.js, MySQL Server y MySQL Workbench.
3. Verificar que el servicio MySQL esté activo y que exista la base autorepuestos_db con sus seis tablas.
4. Configurar en el backend las credenciales de acceso a MySQL mediante la configuración local correspondiente; no publicarlas en el repositorio.
5. En Firebase Console, disponer de un proyecto con Authentication → Email/Password habilitado y el archivo google-services.json de ese proyecto dentro del módulo app.
6. Instalar dependencias del backend con npm.cmd install si se trabaja en PowerShell con restricciones de scripts, y ejecutar node server.js desde su carpeta.
7. En Android Studio, sincronizar Gradle y compilar con .\gradlew.bat :app:assembleDebug desde la raíz del proyecto.
8. Ejecutar en el emulador y verificar que Retrofit apunte a http://10.0.2.2:3000/.
9. Iniciar sesión con un cliente o administrador de prueba y verificar catálogo, inventario y pedidos.

Para un dispositivo físico conectado por Wi-Fi se debe usar una dirección IP accesible del equipo servidor, revisar el firewall y actualizar la URL base. No se debe suponer que 10.0.2.2 funcionará fuera del emulador.

10. Validaciones, errores y seguridad

- Registro: campos obligatorios, formato de correo, contraseña mínima de seis caracteres, coincidencia de contraseñas y mensajes de cuenta existente.
- Interfaz: botón de registro deshabilitado durante la operación y aviso al usuario si falla la sincronización con MySQL.
- Red: manejo de errores HTTP y de conexión; la aplicación requiere que el servidor esté encendido para consultar datos.
- Pedidos: validación de carrito vacío, cantidades y disponibilidad de inventario en el servidor.
- Sesión: cierre de sesión de Firebase y vaciado del carrito en el flujo de salida del cliente.
- Limitación local: API HTTP sin cifrado y encabezado de identidad no verificado criptográficamente. Antes de producción usar HTTPS, verificación de tokens, autorización robusta y secretos protegidos.

11. Pruebas de aceptación recomendadas

Prueba	Resultado esperado

Registro de cliente	Cuenta creada en Firebase y fila con rol cliente en MySQL
Inicio de sesión	Acceso a pantalla según rol
Consulta de productos	Lista consistente con MySQL
Búsqueda/categoría	Resultados filtrados correctamente
Carrito vacío	Mensaje y sin pedido nuevo
Pedido válido	Registro en pedidos/detalle y descuento de stock
Cancelar pedido	Estado cancelado y stock restaurado
Editar producto	Cambios reflejados en inventario y catálogo
Desactivar producto	No aparece en catálogo cliente; sí en administración
Cerrar sesión	Retorno a login

12. Consultas SQL de verificación

```
USE autorepuestos_db;

SELECT id_usuario, nombre, correo, rol FROM usuarios ORDER BY id_usuario DESC;

SELECT p.nombre AS producto, i.stock FROM productos p INNER JOIN inventario i ON
i.id_producto=p.id_producto ORDER BY p.id_producto;

SELECT id_pedido, id_usuario, total, estado FROM pedidos ORDER BY id_pedido DESC;
```

13. Mantenimiento y posibles mejoras

- Respaldar periódicamente la base MySQL y conservar los scripts del esquema.
- Documentar las variables de entorno del backend sin exponer contraseñas.